

SYSTEM REPORT

MedThru

An NFC-carried emergency health record: a patient's card resolves in under five seconds to the facts a first responder needs, with a full clinical history behind it for an authenticated clinician.

Codename NFC-MedConnect **Prepared** 20 September 2026

Scope current implementation, verified against source

16 SQLite tables	52 API endpoints	47 Flutter source files	3 access tiers — public / patient / clinician
----------------------------	----------------------------	-----------------------------------	--

THE PROBLEM

Closing the golden-hour information gap

When someone arrives at an emergency department unconscious or unable to speak, the facts that would let a clinician treat them safely — blood type, a severe allergy, an active blood thinner — usually exist somewhere. They're just unreachable in the minutes that matter.

MedThru is a tri-part system built around one design decision: **the card carries a key, not the data**. Each patient's NFC card holds a 256-bit random token, not a medical record. The server stores only the token's SHA-256 hash. A tap resolves the token to a live record on the backend — so the card never goes stale, a lost card can be revoked without touching the patient's data, and a stolen card leaks nothing on its own.

The card

An NTAG215 tag holding a random token, written as a standard NFC URI record. Any phone's stock NFC reader can open it — no app required.

The app

A Flutter client where patients log vitals and medical history day to day, and clinicians manage patients, appointments, and records.

The backend

A Node/Express API over SQLite — token resolution, access control, and every read or write logged to an audit trail.

The reader bridge

A small Node service that watches a USB NFC reader, extracts the token from a tap, and pushes it live to the portal over SSE.

ACCESS MODEL

What a tap reveals — and to whom

Access is enforced in the API, not the interface. No client can grant itself a view the server won't serve.

- 1 Card taps the reader.** The hardware UID is logged but never trusted as a credential — it's broadcast in the clear and trivially cloned. The token inside the tag's data blocks is what gets checked.
- 2 Layer 1 opens instantly, no login.** Blood type, severe allergies, active medications, and emergency contacts render at `/emergency/:token` — a plain public page, openable by tapping the card with any phone.
- 3 The access is logged.** Every emergency view writes an `audit_log` row before anything is shown.
- 4 A clinician authenticates for Layer 2.** A JWT-bearing login unlocks the full record — history, trends, documents, the ability to write clinical data — scoped to that one patient by the resolved token, so no card can ever reach another patient's rows.

DATA MODEL

Sixteen tables, three groups

Every clinical row — a reading, an allergy, a document — carries a `source` column the server sets to `doctor` or `patient` based on who authenticated the request. A self-reported number is never silently promoted to clinician-verified.

Identity & access

TABLE	HOLDS
<code>patients</code>	Core demographic and emergency-summary fields (blood type, legacy allergy/condition text, DOB, gender)
<code>doctors</code>	Clinician accounts — name, license number, email, bcrypt password hash
<code>cards</code>	One row per physical card ever issued; only the token's SHA-256 hash and a 4-character preview, never the raw token
<code>audit_log</code>	Every access — patient self-view or clinician lookup — with actor, action, and timestamp

Clinical record

TABLE	HOLDS
<code>readings</code>	Vitals time series — blood sugar, blood pressure, cholesterol, etc., each tagged by source
<code>allergies</code>	Allergen, reaction, severity — feeds the Layer 1 emergency page directly
<code>medications</code>	Name, dosage, frequency, start/end dates — active medications feed Layer 1
<code>vaccinations</code>	Vaccine, dose number, date given, next dose due
<code>medical_history</code>	Diagnoses with status and an optional body-region tag for an anatomical view
<code>emergency_contacts</code>	Multiple contacts per patient, each with a relationship
<code>lab_results</code>	Test name, value, unit, reference range, in/out-of-range status
<code>documents</code>	Uploaded files (scans, referral letters) — metadata in the DB, bytes under <code>uploads/</code>

Scheduling & communication

TABLE	HOLDS
clinics	Facilities a patient can book into — hours, slot length, open weekdays
appointments	Bookings with a status lifecycle (requested → confirmed → completed, or rejected/cancelled/reschedule)
messages	A per-patient thread shared between patient and clinician

CLINICAL PORTAL

What a clinician can do

CAPABILITY	DETAIL	
Dashboard	Today's schedule, patient-growth and prescription-volume KPIs against the prior period, age/gender demographics, this month's completed vs. cancelled visits, newest registrations	• Built
Practice reports	Appointment volume across the last 8 weeks, most common diagnoses, status breakdown	• Built
Patient directory & summary	Search patients; open a full record with vitals trends, records, documents	• Built
Card issuance	Issue a token at registration (shown once), reissue (auto-revokes the old card in the same transaction), or revoke outright — reissue/revoke restricted to admins	• Built
Appointment queue	Confirm, reject, cancel, or complete a booking; state transitions are enforced server-side	• Built
Reschedule review	Offer a patient AI-ranked alternative slots when a booking needs to move	• Built

Record entry	Add allergies, medications, vaccinations, history, lab results, notes — attributed as clinician-sourced	● Built
Audit trail	Full access history per patient — who looked, and when	● Built
Messaging	A threaded conversation per patient, polled live	● Built
Physician verification stamp	Promoting a patient-entered item to clinician-verified, distinct from source attribution	● Planned

PATIENT APP

What a patient can do

CAPABILITY	DETAIL	
Daily logging	Record vitals and readings; trend charts collapse same-day entries into a daily band so noise doesn't drown the pattern	● Built
Health record management	Add allergies, medications, vaccinations, medical history, lab results, emergency contacts	● Built
Documents	Upload and view scans, referral letters, and other files	● Built
Appointments	Book by clinic and available slot, view on a calendar, cancel, or request a reschedule	● Built
Card-free login	Register a phone number + password once while holding the card, then log in without it later	● Built
Messaging	Message the treating clinician directly	● Built
PDF export	One-page emergency summary (allergies + medications), built and shared from the phone	● Built
Calendar export	Save a booked appointment as an .ics file for any calendar app	● Built
Reminders	Local, on-device notifications for medications and appointments — deliberately never synced to the server	● Built

Activity recap	A day-by-day view of how consistently readings are being logged	● Built
Care plan & clinical updates	Read-facing views of what the clinician has prescribed or noted	● Built

AI INTEGRATION

Where AI sits in the workflow

AI is scoped to decision *support*: advisory, attributable, and never a replacement for the underlying data.

Reschedule assistant — built. When a booking needs to move, a deterministic scorer (`reschedule.js`) ranks candidate slots on two independent 0–100 axes: how well a slot fits the patient (closeness to their original time, urgency inferred from the stated reason) and how well it fits the clinic's day (fills a gap rather than fragmenting it, keeps load near what's typical). Gemini (`gemini-flash-lite-latest`) is then asked only to *re-order and caption* that fixed set with a short, reassuring rationale — it can never add, drop, or invent a slot; the server verifies the returned set is an exact permutation before trusting the order, and falls back silently to the deterministic ranking if Gemini is slow, unreachable, or unconfigured.

Two further features are specified in the project's design documents but not yet present in the codebase:

- **Clinical Executive Summary** — a three-bullet synthesis of 30 days of vitals, surfacing trends like a hypertensive spike or glycaemic volatility that are invisible in a raw table. ● Planned
- **Triage contraindication engine** — cross-referencing a proposed treatment against a patient's logged allergies and active medications to flag a dangerous interaction before it happens. ● Planned

SECURITY

Threat model and mitigations

THREAT	MITIGATION	
Cloned card via broadcast UID	UID is logged only; the credential is the token in the tag's data blocks	● Built
Token database leak	Only a SHA-256 hash is stored; the raw token is returned once, at issue	● Built
Card lost or stolen	Revoke and reissue in one transaction; only one active card per patient	● Built
Token guessing / enumeration	256-bit token space plus a per-route lookup rate limit	● Built
Clinician password guessing	bcrypt hashing plus a login rate limit	● Built
Cross-patient data access	Every token-scoped query is filtered by the resolved <code>patient_id</code> in SQL, not just checked in the UI	● Built
Unaudited access	Every read and write — including a patient's own — writes an <code>audit_log</code> row	● Built
Hardcoded JWT secret	Falls back to a dev placeholder if <code>JWT_SECRET</code> isn't set in the environment	● Open
Data at rest	Database is currently unencrypted on disk	● Open

Pre-deployment blocker: the JWT secret and at-rest encryption gaps are the two items that must close before this system touches a real patient's data. Everything else in the threat model above is already closed.

HARDWARE

The physical loop

An ACR122U USB reader (13.56 MHz, PC/SC) watches for a tap. `tag-storage.js` writes the token as a standard NFC Forum URI record pointing at the emergency page — meaning a patient's card is readable by any stock phone, not only MedThru's own bridge; a legacy raw-byte format is still read for cards provisioned

before that scheme existed. The bridge cold-starts the backend and client if neither is already running, so the operator's only action at a terminal is presenting the card. `simulate-tap.js` exercises the entire path without any hardware attached, which is how the software has been validated ahead of the physical reader's arrival.

STATUS

Built, open, and next

Solidly built

Full record CRUD across 16 tables, card issuance and lifecycle, dual-layer access enforcement, audit logging, appointment booking with conflict-safe slotting, the AI-assisted reschedule flow, and a 47-file Flutter client for both patient and clinician roles.

Open before real data

Replace the hardcoded JWT fallback secret with a required environment value, and add encryption at rest.

Designed, not yet built

The Clinical Executive Summary and contraindication engine; the explicit physician verification stamp on top of source attribution.

Longer term

A web portal in place of the Flutter desktop client; an iOS build (architecture is cross-platform already, blocked on toolchain access); HL7 FHIR export for interoperability with national infrastructure.

Compiled directly from the current source tree — `server.js`, `db.js`, `auth.js`, `reschedule.js`, `gemini.js`, `reader.js`, `tag-storage.js`, and the 47 files under `app/lib/`.